02: Intro to R for Regression

Stat 230 - Spring 2026

> **Note**
>
> To follow the tutorial version in the browser, you can use https://aluby.github.io/stat230/activities/02-r-live

1 Overview

In Stat 230 we will use the R statistical programming language to visualize our data and fit our regression models. Many of you have seen R before, but if you are coming straight from AP statistics this might be your first time using R. Today our goal it to learn the basics of writing and running code in R. To do this, we'll run our code in a tutorial webpage. Next class I'll introduce you to the RStudio server and how to create R Markdown files.

2 Loading data

Today we will work with the `movies` data set. The data set contains the critic's score and the audience score for movie released in 2014-2015. The critic's score is the percentage of critics who had a favorable view of the movie. Similarly, the audience score is the percentage of users who had a favorable view of the movie. Today, you'll explore the relationship between the audience and critic scores.

To load data into R, we use the `read.csv()` command and pass in either a file path or URL in quotes. Below, the data are being loaded from the web. Click "Run code" to load the data.

```
movies <- read.csv("https://aloy.github.io/stat230-materials/data/movie_scores.csv")
```

Notice that we use parentheses with functions and place the arguments for that function inside those parentheses.

> **Note**
>
> You should notice that R did not *appear* to do anything when you ran the above code. That's because everything was done in the background and R will only show you what is requested. So don't panic if you don't see any output!

3 Getting a glimpse of the data set

Throughout the term I'll refer to small pieces of code as "code chunks." This is both descriptive and will be what RStudio (our interface) calls them later on. For today, we'll focus on how to write small code chunks to accomplish specific tasks for simple linear regression.

For example, we might wish to get an overview of the data set by printing the first few rows via `head()`

```
head(movies)
```

I prefer to use a different function, `glimpse()`, which is in the {dplyr} package. To load a package we first must run the `library()` command. The below code chunk loads this package and gives a "glimpse" of the `movies` data set:

```
library(dplyr)
glimpse(movies)
```

3.1 Checkpoint questions:

Using the output from the above code chunk, answer the following questions in the code box below:

- How many rows are in the `movies` data set?
- How many variables are in the `movies` data set? Are any categorical? Are any quantitative?

4 Univariate exploration

Before jumping into a regression model, it can be useful to understand more about each variable. This can be accomplished using intro stat tools such as histograms and summary statistics.

The fastest way to get started is using the `summary()` function, which returns a brief set of summary statistics for each variable in the data set.

```
summary(movies)
```

R also has built-in functions to compute summary statistics one by one for a specific column. To "extract" a variable from the data set, we use a `$`. For example, we can extract the `critics` column from the `movies` data frame and calculate the standard deviation:

```
sd(movies$critics)
```

Other useful summary statistics for quantitative variables include (here `x` denotes an extracted column):

Function	Summary statistic
`median(x)`	median
`mean(x)`	mean
`var(x)`	variance
`IQR(x)`	interquartile range

To create graphics in this course, I'll often use the {ggformula} R package. All of the functions will begin with `gf_` and have similar syntax. If you're familiar with the {ggplot2} package, feel free to use it for this class, but many students like the "one-liners" that {ggformula} allows.

To begin, load the {ggformula} package using the library command:

```
library(ggformula)
```

For example, to create a histogram of the critic's score we can run

```
#| fig-alt: "Histogram of critics."
gf_histogram(~critics, data = movies, bins = 10)
```

Here we use the `~` to tell R that `critics` is a variable in the data set. We must also specify `data` and adjust the number of `bins` as needed.

Similar syntax is used to create a boxplot

```
#| fig-alt: "Boxplot of critics."
gf_boxplot(~critics, data = movies)
```

4.1 Checkpoint questions

Use the below code box for the checkpoint questions:

```
#| min-lines: 5
```

- Calculate the mean audience score using the `mean()` function.
- Create a histogram of the `audience` score and describe what you see.
- Change the number of bins for a histogram of `audience`. What number of bins seems "about right" in your opinion?

5 Bivariate exploration

5.1 Scatterplots

In this course we are interested in exploring relationships between variables. If we have two quantitative variables, then a scatterplot is a natural way to visually explore this relationship. To create a scatterplot we can use the `gf_point()` function (since scatterplots are just points):

```
#| fig-alt: "Scatterplot with critics on the x-axis and audience on the y-axis."
gf_point(audience ~ critics, data = movies)
```

Notice that we put the response variable on the left of the ~ and the explanatory variable on the right.

5.1.1 Checkpoint questions

Use the below code box for the checkpoint questions:

```
#| min-lines: 5
```

- Create a scatterplot where `critics` score is the response variable and `audience` score is the explanatory variable.
- You can adjust the axis labels by adding an `xlab` or `ylab` argument. Add `xlab = "Your axis label"` and `ylab = "Your other axis label"` to your scatterplot and give more verbose axis labels.

5.2 Scatterplots with regression lines

You can also add a simple linear regression line to your scatterplot by adding a **layer**. To add a layer, we append the |> pipe operator to the end of our `gf_point()` code and use the `gf_lm()` command to add a line:

```
#| fig-alt: "Scatterplot with critics on the x-axis and audience on the y-axis, with
gf_point(audience ~ critics, data = movies) |>
  gf_lm()
```

5.3 Correlation

If you want a numeric summary of the strength of the linear association between two quantitative variables, then you can calculate the correlation. In R, you do this using the `cor()` function. Similar to calculating the mean, you need to extract the columns using $ as shown below:

```
cor(movies$audience, movies$critics)
```

5.3.1 Checkpoint questions

Use the below code box for the checkpoint question:

```
#| min-lines: 5
```

- Swap the order of the variables in the `cor()` command. Does the correlation change? Is this what you expected?

6 Fitting a regression model

Finally, lets fit a simple linear regression model. To do this we use the `lm()` command (which stands for linear model). Here, we use the same syntax as when creating a scatterplot:

```
lm(audience ~ critics, data = movies)
```

When you run this command, R prints some very basic information about the SLR model.

Checkpoint: What information do you get when you run the above command?

Often, you will want to obtain more information than you get from simply printing the fitted model. The `summary()` function provides much more information. I recommend storing your fitted model as a named object (such as `movie_lm`) and then running the `summary()`:

```
movie_lm <- lm(audience ~ critics, data = movies)
summary(movie_lm)
```

6.1 Checkpoint question

What information do you get when you run the above `summary()` command? If you're not sure what something means, discuss it with your group and then ask about it in our debrief.

7 Constructing confidence intervals

The table obtained by `summary()` contains all of the necessary information to conduct hypothesis tests for regression coefficients; however, it does not provide confidence intervals. To instruct R to construct a confidence interval for a regression coefficient, we can use the `confint()` function:

```
confint(movie_lm, level = 0.95)
```

7.1 Checkpoint questions

- What information do you get when you run the above `confint()` command?
- Change the `level` argument to construct a 90% confidence interval. How does the interval change?

8 Making predictions

To make predictions, we use our fitted regression equation. While you can (and should be able to) do this by hand, you can also use R as your calculator.

Suppose we can use our model to predict the audience score for more recent movies. Then we could predict the audience score for the Barbie Movie by running:

```
barbie_movie <- data.frame(critics = 88)
predict(movie_lm, newdata = barbie_movie)
```

Here, we first need to create a new data set with a column named identically to the original data set. Here we have a single column named `critics` with the value of the explanatory variable we want to use. Next, we use the `predict()` function, passing in the name of our model and the new data set we want to make predictions for.

8.1 Checkpoint questions

Use the below code box for the checkpoint questions:

```
#| min-lines: 5
```

- The critics score for Asteroid City was 75. What is the expected audience score for this movie?
- The actual (observed) value for Asteroid City was 62. Did the mode over- or under-predict the audience score?
- Do you think it's reasonable to use our model to predict the audience score for movies released in 2023? Why or why not?

8.2 Making multiple predictions

If you want to make multiple predictions, then you can create a new data frame with multiple values for the explanatory variable. For example, we can use `c(88, 75)` to make a data frame with two values in the critics column:

```
new_movies <- data.frame(critics = c(88, 75))
predict(movie_lm, newdata = new_movies)
```

---

9 Function quick reference

The following table summarizes the functions we learned today:

Function	Purpose
`read.csv("file_path_or_URL")`	Load a CSV data set from a file or the web
`head(data)`	Print the first few rows of a data set
`glimpse(data)`	Get a quick overview of a data set (from {dplyr} package)

Function	Purpose
`summary(data)`	Get summary statistics for each variable in a data set
`mean(x)`	Calculate the mean of a quantitative variable
`median(x)`	Calculate the median of a quantitative variable
`sd(x)`	Calculate the standard deviation of a quantitative variable
`var(x)`	Calculate the variance of a quantitative variable
`gf_histogram(~x, data = data, bins = n)`	Create a histogram of a quantitative variable (from {ggformula} package)
`gf_boxplot(~x, data = data)`	Create a boxplot of a quantitative variable (from {ggformula} package)
`gf_point(y ~ x, data = data)`	Create a scatterplot of two quantitative variables (from {ggformula} package)
`gf_lm()`	Add a simple linear regression line to a scatterplot (from {ggformula} package)
`cor(x, y)`	Calculate the correlation between two quantitative variables
`lm(y ~ x, data = data)`	Fit a simple linear regression model
`summary(model)`	Get detailed information about a fitted regression model
`confint(model, level = 0.95)`	Construct a confidence interval for regression coefficients
`predict(model, newdata = new_data)`	Make predictions using a fitted regression model for new data

---

10 Acknowledgements

The movie example was adapted by Adam Loy from Maria Tackett's draft of *Introduction to Regression Analysis: A Data Science Approach*.